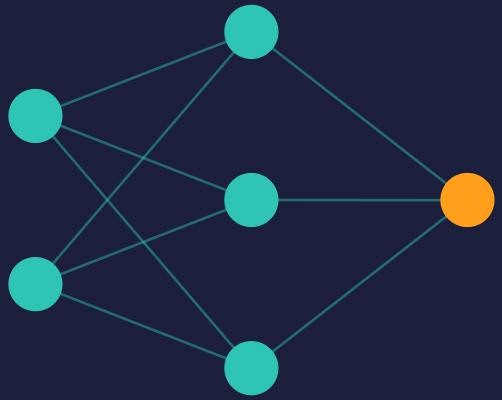
دوره یادگیری عمیق و شبکه‌های عصبی



جلسه ۱۳

Transformer از صفر

Self-Attention، Multi-Head Attention، Positional Encoding و یک GPT کوچک

کیوان محمودی

در این جلسه

۱ چرا Transformer؟

۲ Query، Key و Value

۳ Scaled Dot-Product Attention

۴ ماسک علّی

۵ Multi-Head Attention

۶ Positional Encoding

۷ بلوک Transformer

۸ یک GPT کوچک و تولید متن

Attention Is All You Need (2017)

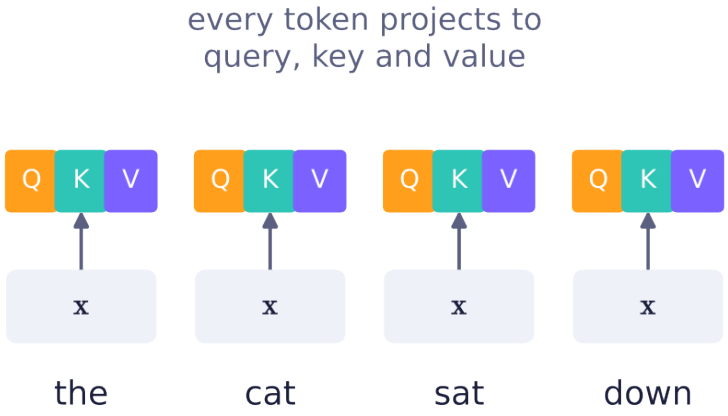
مشکلات RNN

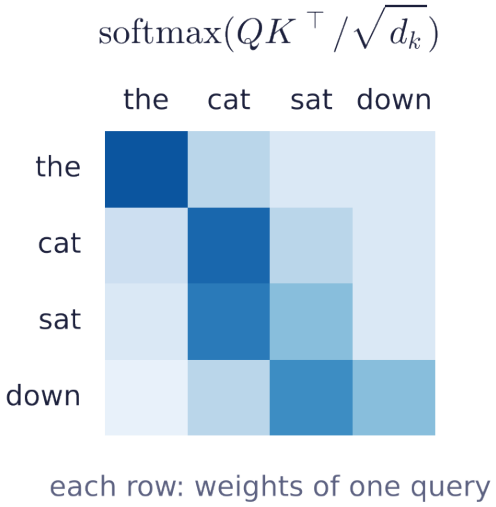
- پردازش ترتیبی: گام t منتظر گام قبلی است.
- موازی‌سازی روی GPU محدود است.
- مسیر طولانی بین Tokenهای دور

Transformer

- حذف کامل RNN؛ فقط Self-Attention
- هر Token مستقیماً به همه Tokenها نگاه می‌کند.
- همه مکان‌ها همزمان پردازش می‌شوند.
- پایه BERT، GPT، T5، LLaMA و Claude

Self-Attention: Query، Key و Value





Query: دنبال چه هستم؟ Key: چه دارم؟ Value: چه تحویل می‌دهم؟ شباهت Query و Key وزن هر Value را تعیین می‌کند

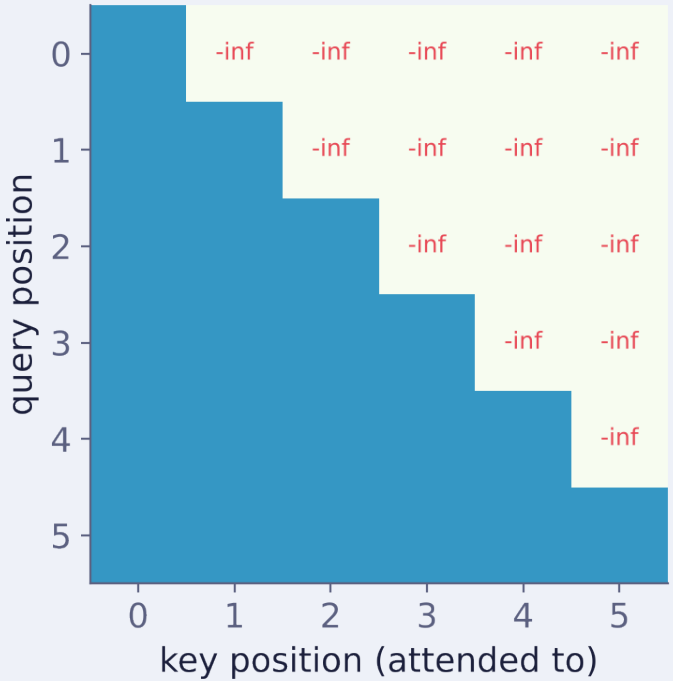
Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$$Q = XW_Q \qquad K = XW_K \qquad V = XW_V$$

- هر سه با ضرب ورودی در ماتریس‌های قابل آموزش ساخته می‌شوند.
- واریانس ضرب داخلی برابر بعد است؛ بدون تقسیم بر ریشه آن، Softmax اشباع می‌شود.
- با بعد ۵۱۲: بیشینه وزن بدون مقیاس تقریباً ۱، با مقیاس بسیار کمتر

ماسک علّی (Causal Mask)



- مدل زبانی Token بعدی را پیش‌بینی می‌کند.
- نگاه به آینده یعنی دیدن جواب: تقلب!
- امتیازهای بالای قطر اصلی منفی بی‌نهایت می‌شوند.
- بعد از Softmax وزن آینده صفر است.
- یک Forward روی طول T، همزمان T مسئله پیش‌بینی را آموزش می‌دهد.

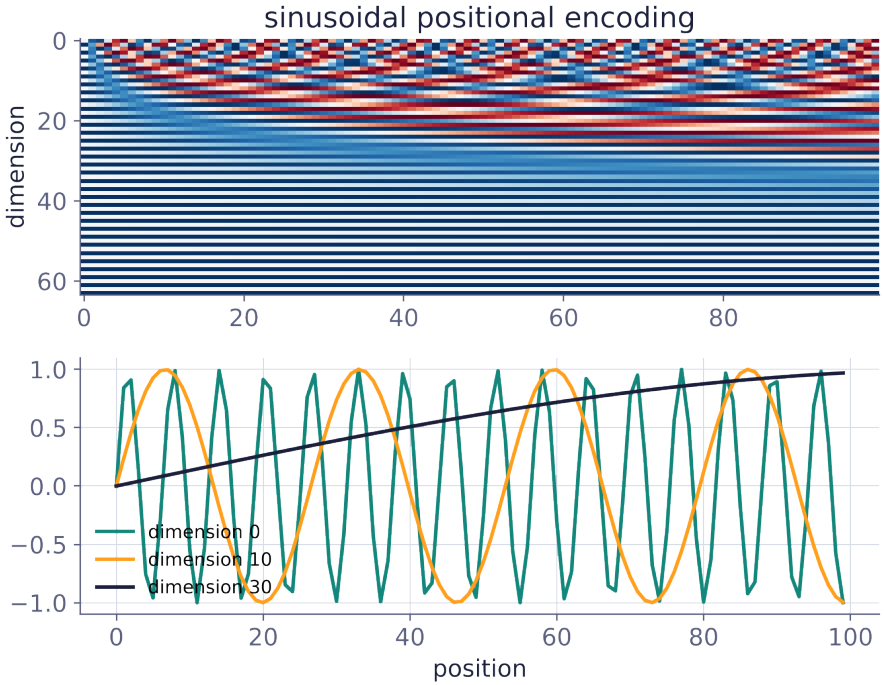
Multi-Head Attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.h, self.d = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)   # all heads at once
        self.proj = nn.Linear(d_model, d_model)      # W_O

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)
        q, k, v = [t.view(B, T, self.h, self.d).transpose(1, 2)
                   for t in (q, k, v)]               # (B, h, T, d)
        mask = torch.triu(torch.ones(T, T, dtype=torch.bool), 1)
        out, _ = scaled_dot_product_attention(q, k, v, mask)
        out = out.transpose(1, 2).contiguous().view(B, T, C)
        return self.proj(out)
```

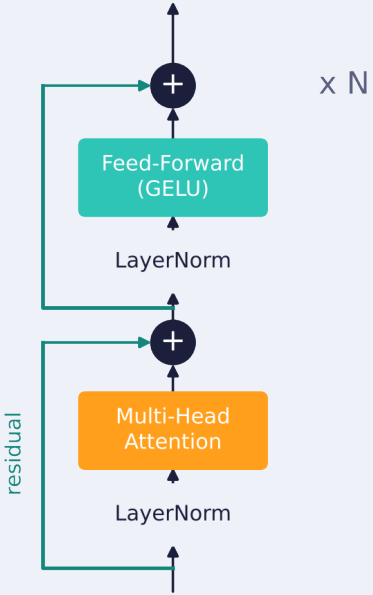
- چند Attention موازی در زیرفضاهای کوچک‌تر
- هر Head یک نوع رابطه را دنبال می‌کند.
- خروجی Headها کنار هم گذاشته و با W_O ترکیب می‌شوند.

Positional Encoding



- Attention به ترتیب حساس نیست!
- اطلاعات موقعیت باید به ورودی اضافه شود.
- سینوسی (مقاله اصلی): هر بعد یک موج با فرکانس متفاوت
- قابل آموزش (GPT و BERT): یک nn.Embedding برای موقعیت‌ها
- مدل‌های جدیدتر: RoPE

بلوک Transformer



- Multi-Head Attention: ارتباط بین Tokenها
- Feed-Forward: MLP دولایه روی هر Token جداگانه، با بعد میانی چهار برابر
- اتصال Residual دور هر زیرلایه (ایده ResNet)
- Pre-LN: LayerNorm قبل از هر زیرلایه؛ آموزش پایدارتر
- بلوک‌ها N بار تکرار می‌شوند.

سه خانواده Transformer

۱ فقط Encoder

Attention دوطرفه؛ فهم متن: طبقه‌بندی و NER. مثال: BERT و ParsBERT

۲ فقط Decoder

Attention علّی؛ تولید متن با پیش‌بینی Token بعدی. مثال: GPT و LLaMA

۳ Encoder–Decoder

Cross-Attention بین دو بخش؛ ترجمه و خلاصه‌سازی. مثال: T5 و BART

MiniGPT: یک مدل زبانی در سطح کاراکتر

```
class MiniGPT(nn.Module):
    def __init__(self, vocab, d_model=192, n_heads=6, n_layers=6):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab, d_model)
        self.pos_emb = nn.Embedding(BLOCK_SIZE, d_model)
        self.blocks = nn.Sequential(*[TransformerBlock(d_model, n_heads)
                                      for _ in range(n_layers)])
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab, bias=False)

    def forward(self, idx):
        T = idx.shape[1]
        x = self.tok_emb(idx) + self.pos_emb(torch.arange(T))
        return self.head(self.ln_f(self.blocks(x)))   # (B, T, vocab)
```

- داده: Tiny Shakespeare، حدود یک میلیون کاراکتر
- طول زمینه ۱۲۸ و حدود ۲/۷ میلیون پارامتر
- AdamW با Warmup و کاهش کسینوسی
- چند دقیقه روی GPU رایگان Colab

تولید متن: Temperature و Top-k

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

- دمای کم: توزیع تیزتر، متن محافظه‌کارتر و تکراری‌تر
- دمای زیاد: توزیع صاف‌تر، متن خلاقانه‌تر ولی پرخطاتر
- Top-k و Top-p: حذف گزینه‌های بسیار نامحتمل
- GPT-3 همین معماری است؛ تفاوت اصلی در مقیاس است.

جمع‌بندی

- Self-Attention: هر Token با Query خود Key همه را می‌سنجد و Valueها را ترکیب می‌کند.
- تقسیم بر ریشه بعد از اشباع Softmax جلوگیری می‌کند.
- ماسک علّی نگاه به آینده را در مدل زبانی ممنوع می‌کند.
- Multi-Head Attention چند رابطه را موازی مدل می‌کند.
- Positional Encoding اطلاعات ترتیب را اضافه می‌کند.
- بلوک: Attention، Pre-LN و FFN، هر کدام با اتصال Residual

تمرین

۱ از F.scaled_dot_product_attention با is_causal=True استفاده و سرعت را مقایسه کنید.

۲ Embedding موقعیت را با Positional Encoding سینوسی جایگزین کنید.

۳ تعداد لایه‌ها و بعد مدل را تغییر دهید و Loss را بر حسب پارامتر رسم کنید.

۴ Top-p Sampling را به متد generate اضافه کنید.

۵ مدل را روی شعر فارسی با مالکیت عمومی آموزش دهید.